iTPU IT PARK UNIVERSITY

ITPU IT PARK UNIVERSITY

Hello! How is it going?

waiting a miracle

nice

za varudo awesome notbad

as usual wonderful

little sleepy and wanna

nigorakhon akhmadjanova

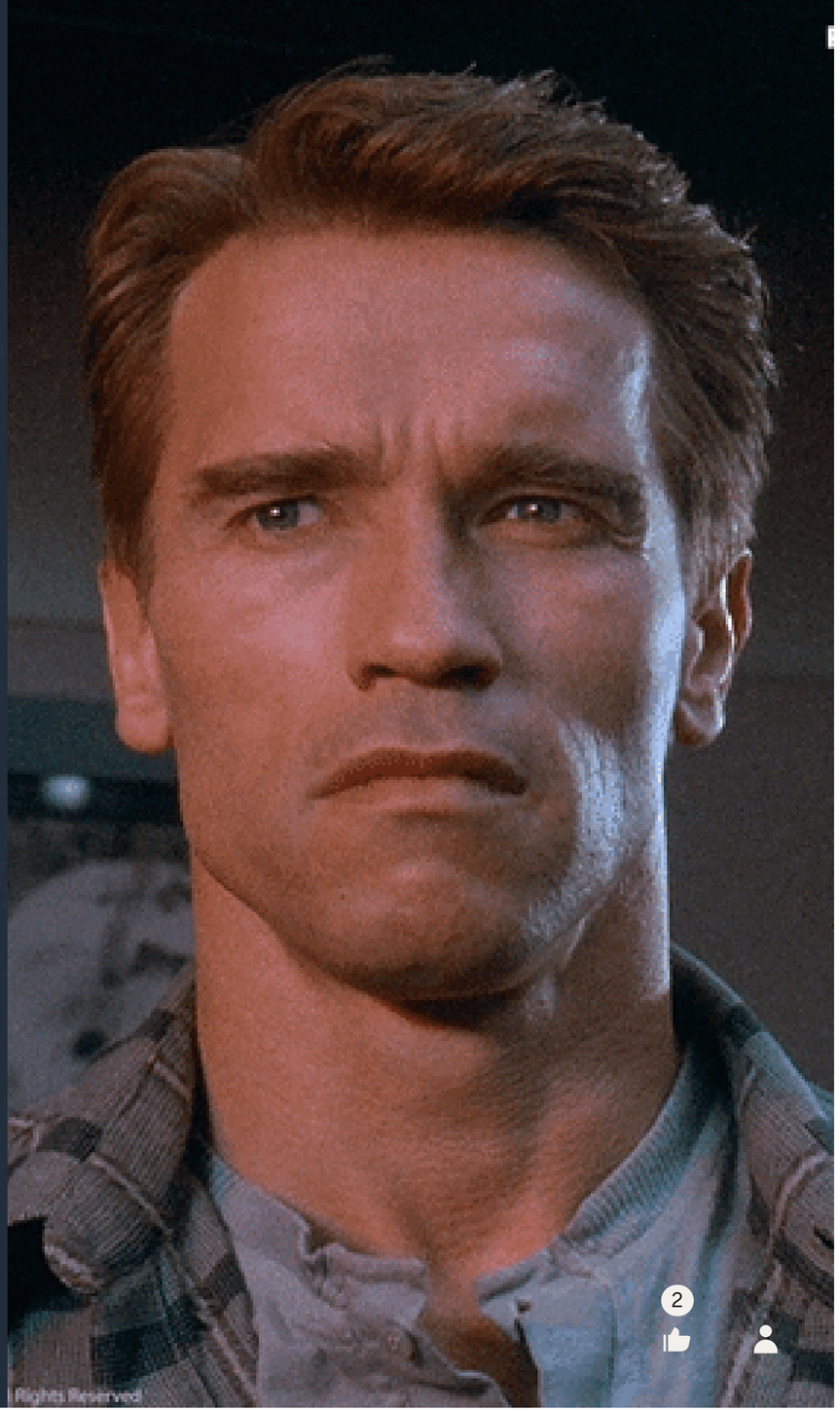
iTPU IT PARK UNIVERSITY

Agenda

- Recall last session
- Discuss current topic
- Practice
- Quiz

IT PARK UNIVERSITY

Total Recall



What do you remember most from few previous workshops? (short answer)

CONNECTION POOL	Connection pool and jdbc	Gennadiy Shegay	jdbc
connection pool	transaction	jdbc	test

IT PARK UNIVERSITY

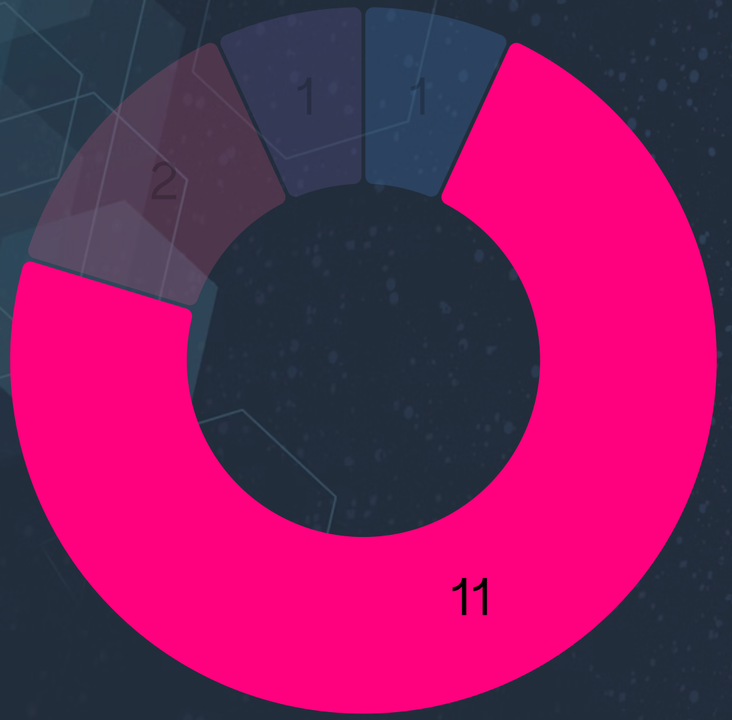
What do you remember most from few previous workshops? (short answer)

connection pool

jdbc & connection pool

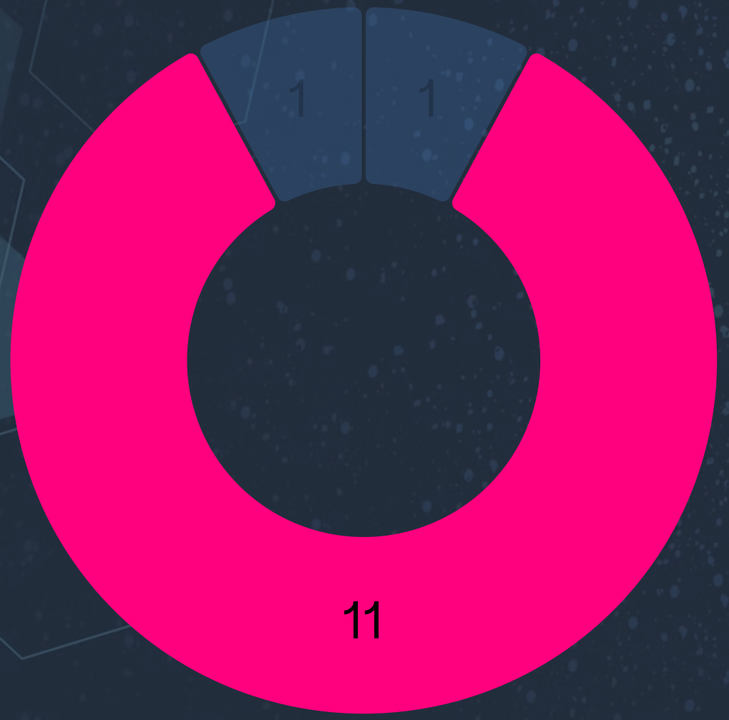
IT PARK UNIVERSITY

What does setAutoCommit(false) do in JDBC?



1	Executes the transaction	×
11	Disables auto-commit mode	✓
2	Commits all transactions immediately	×
1	Rolls back current transaction	×

How do you commit a transaction in JDBC?

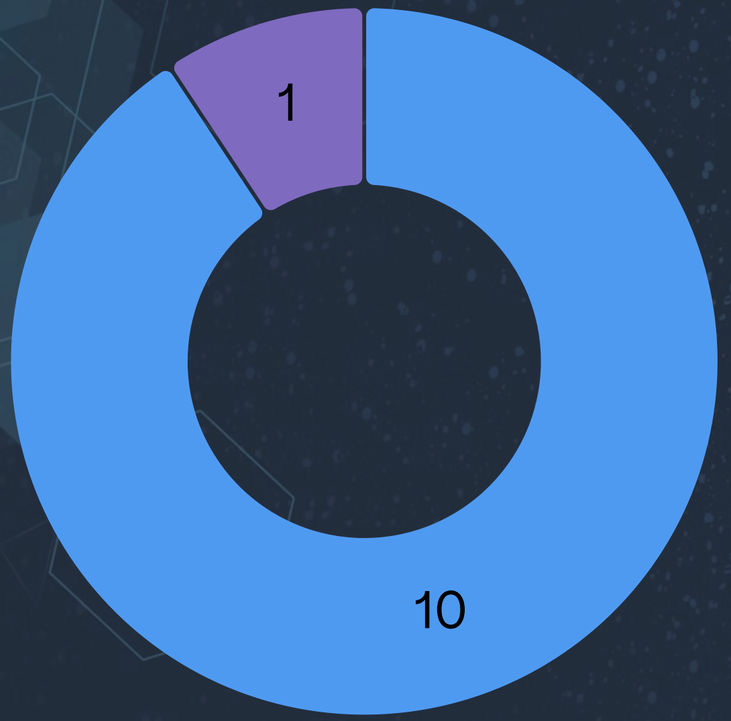


1	AutoCommit()	×
11	conn.commit()	✓
0	conn.rollback()	×
1	commitTransaction()	×

What does rollback() do in JDBC?

0	Executes all SQL statements	×
0	Ends JDBC connection	×
10	Cancels the current transaction	✓
0	Saves the state of the database	×

When should transactions be used in JDBC?



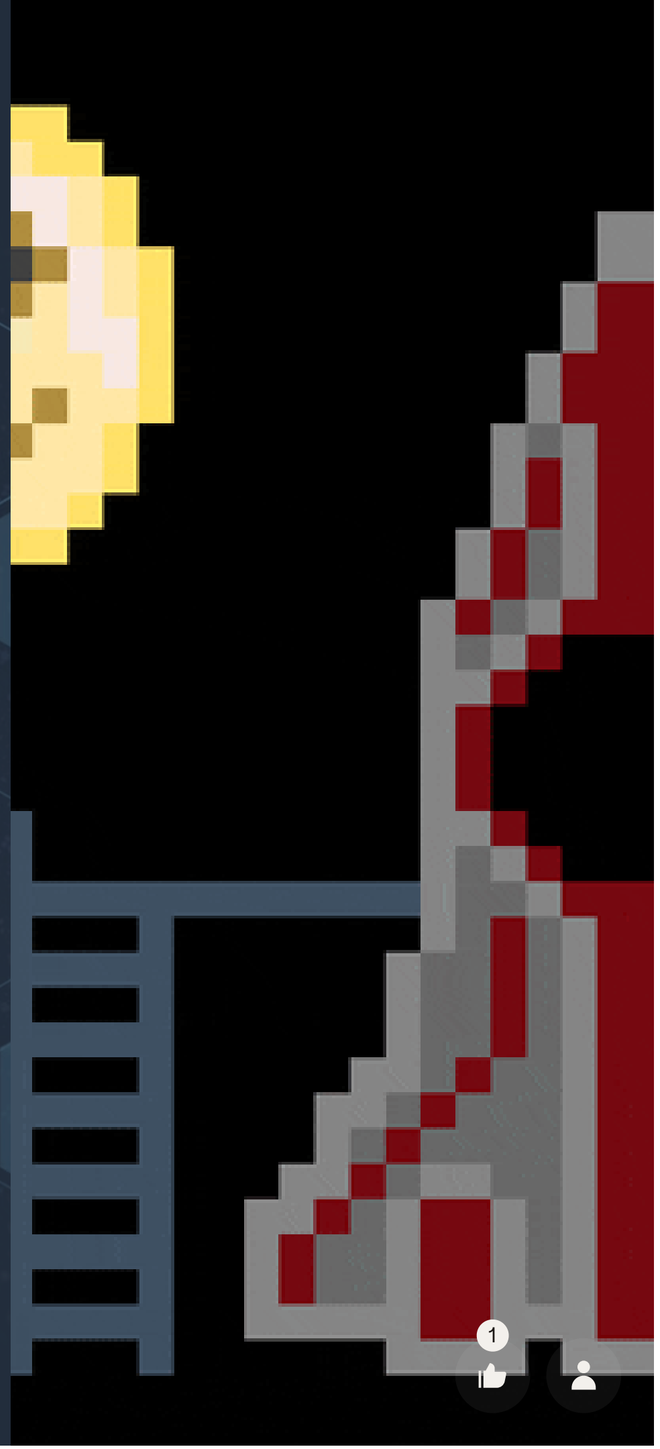
10	For operations requiring atomicity
0	To test database performance
0	For selecting a single row
1	To connect to the database

Next topic

iTPU IT PARK UNIVERSITY

Random fact

There were active volcanoes on the moon when dinosaurs were alive.



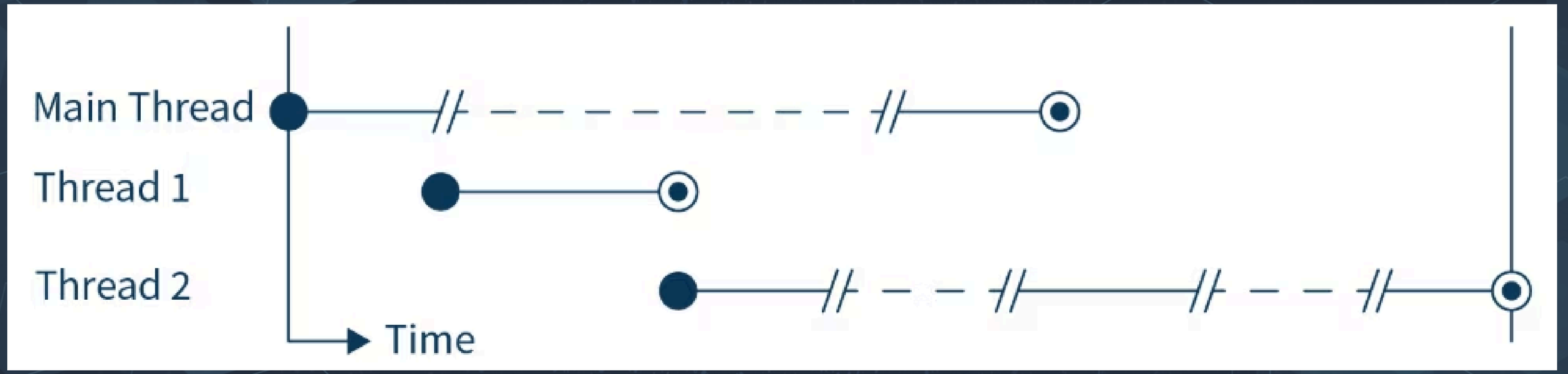
ITPU IT PARK UNIVERSITY

1. Introduction to Threads

Multithreading in parts:

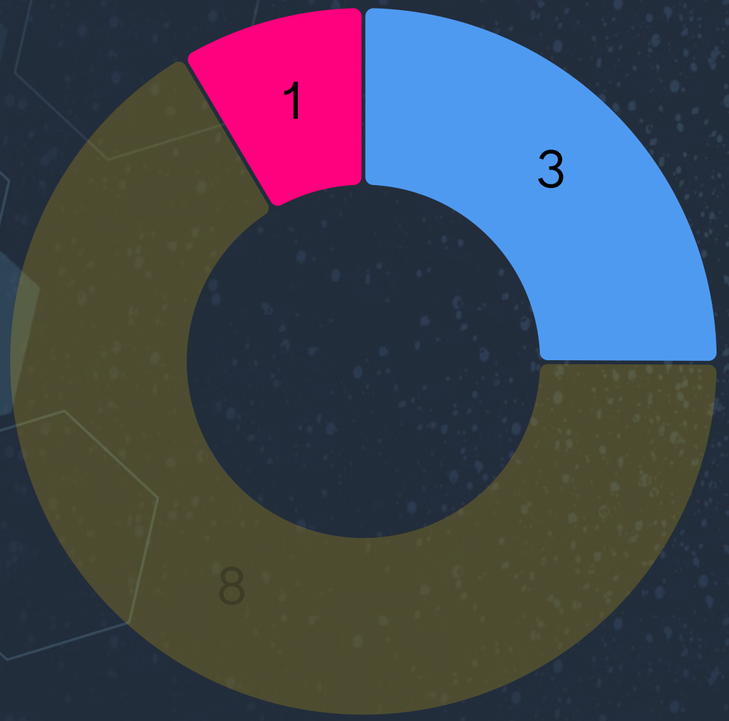
- An operating system allocates the processor's time in parts (quanta) for every process and creates the impression of parallel work.
- Multithread applications broaden the idea of multitasking by adding a bottom level: Separate applications can perform several tasks simultaneously.
- Each task is called a **thread**, meaning a thread of execution. So, applications capable of performing more than one thread are called **multithread**.

- A significant difference between many processes and threads is that every process has its own set of variables, while several threads share the same data.
- Using shared data:
  - Provides more effective interaction between threads
  - Facilitates communication between threads
- Threads allow you to organize tasks so that they are executed simultaneously. Each task is encapsulated in a separate thread.



Process - is an application. Threads are parts of an application that can run independently. Threads can be performed concurrently

I am typing text with my both hands - is it a concurrent or parallel activit



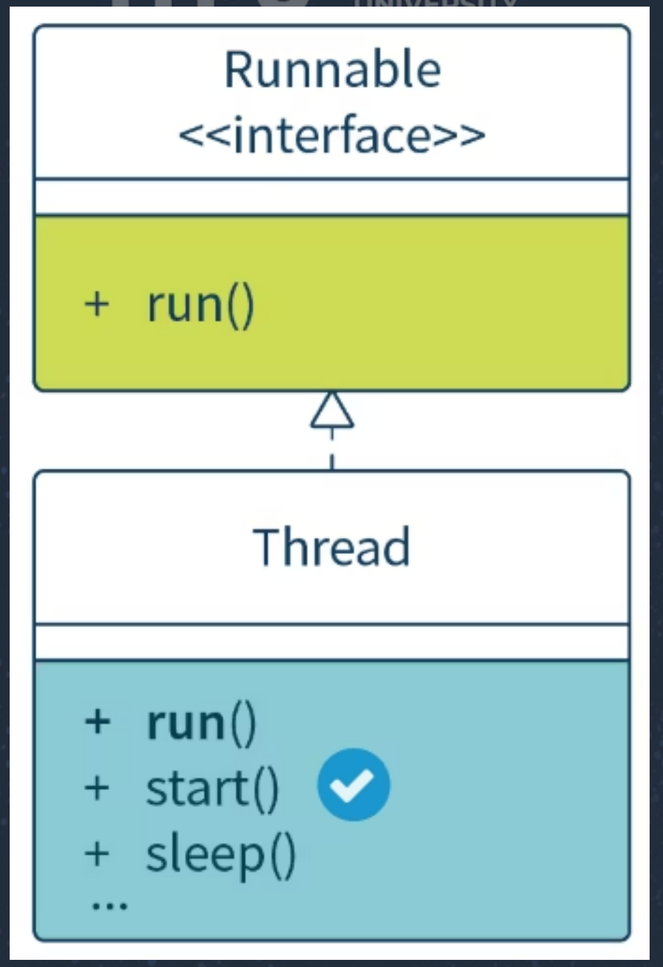
3	concurrent	✓
8	parallel	×
1	concurrent-parallel	✓

Creation and Execution of Threads

There are several ways to create and execute a thread. The first option is by using an extension of the Thread class.

To create and start a thread, follow the steps below:

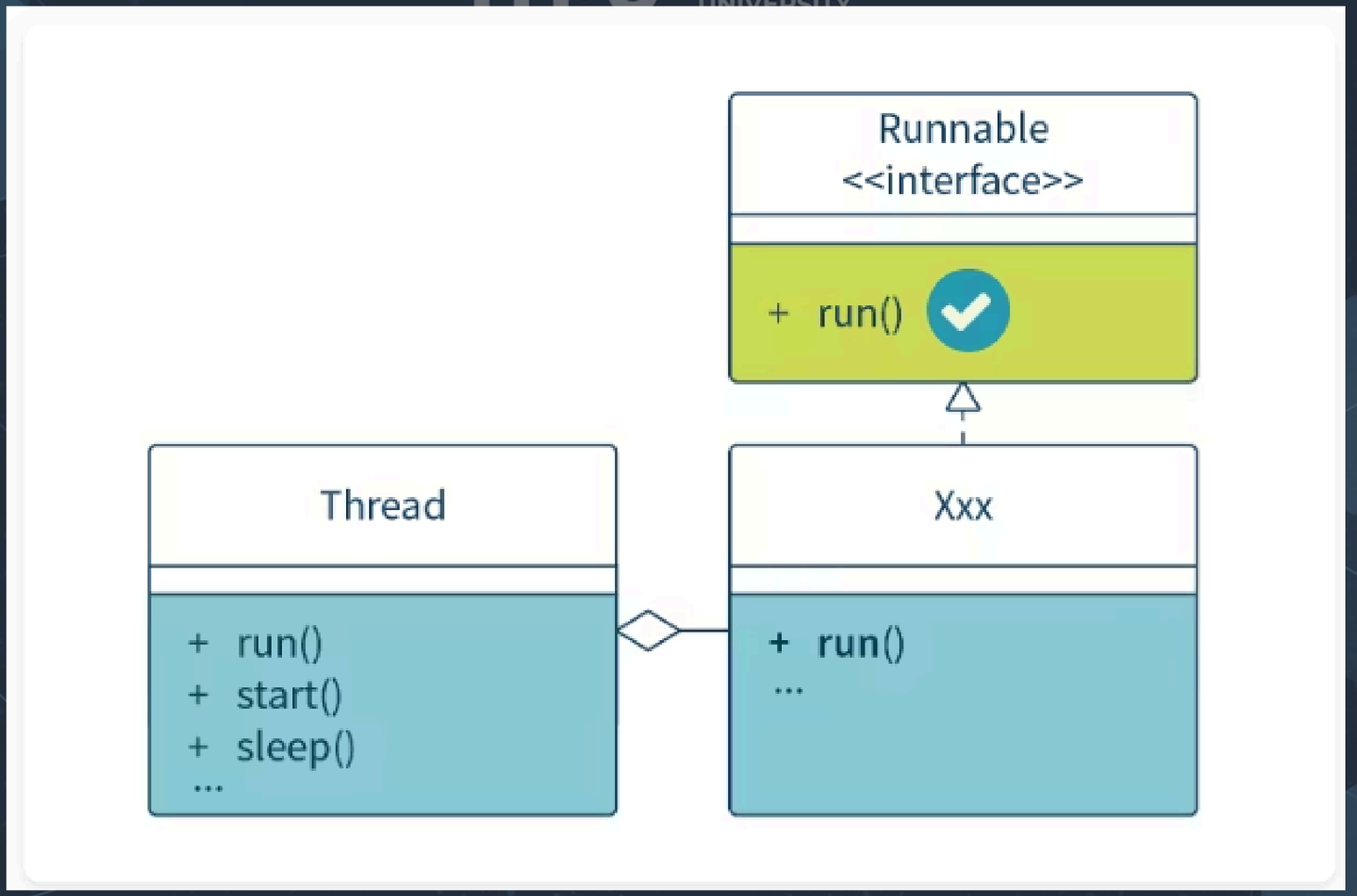
1. Describe the subclass of the Thread class and override the run() method.
2. Create an instance of the subclass.
3. Call a start() method for the object created.



CreateThreadD

Threads Extension

- It is easier to use in simple applications.
- A thread cannot extend any other class.
- Thread control can be overridden (i.e., overriding various methods of the Thread class).



CreateRunnableDemo

Runnable Implementation

- It is more flexible because a thread can extend any other class.
- The task solved by the thread is separate from the code that controls this thread.
- This approach is used in high-level thread management.

iTPU IT PARK UNIVERSITY

Asynchronous Thread Execution

```
public interface Callable<V> {
    V call() throws Exception;
}
```

The body of the thread returns the result of executing it.

java.util.concurrent.Callable

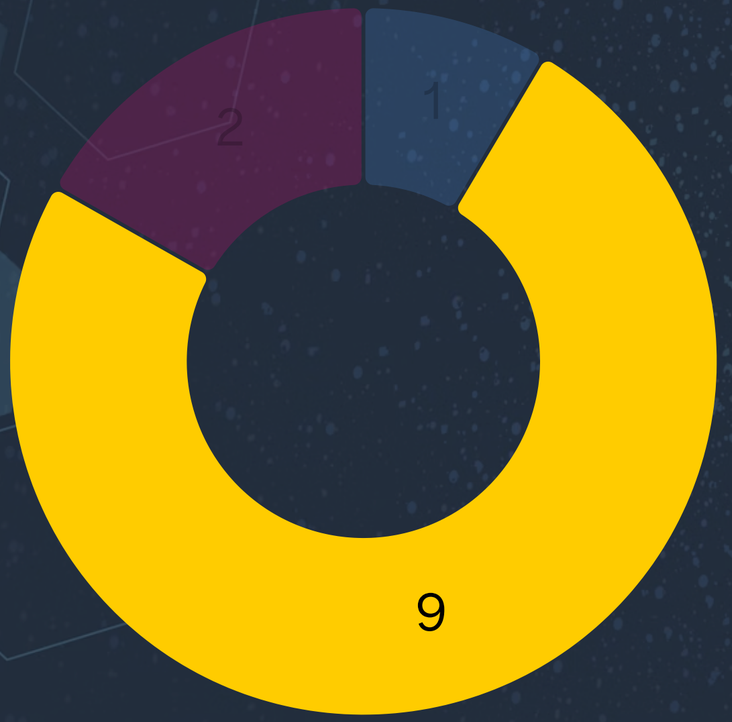
asynchronous execution of a thread that returns a result

Features:

1. It allows any thread that started the Callable thread to get the result of executing this thread.
2. It is generalized—i.e., it defines the type of value returned.
3. It can throw exceptions—both checked and unchecked—without influencing other tasks that are running.

IT PARK UNIVERSITY

I am playing piano with my both hands - is it a concurrent or parallel activity?



1	concurrent	×
9	parallel	✓
2	concurrent-parallel	×

2

12 / 12

IT PARK UNIVERSITY

Real world example of concurrency vs parallelizm

what about items andspels

gennadiy knows

drinking

invisibility and tp scrol

chess

char in dota

shadow fiend

a receptionist concurent

invoker

process executing

channelling spells n item

driving

eyes look at thesamepoint

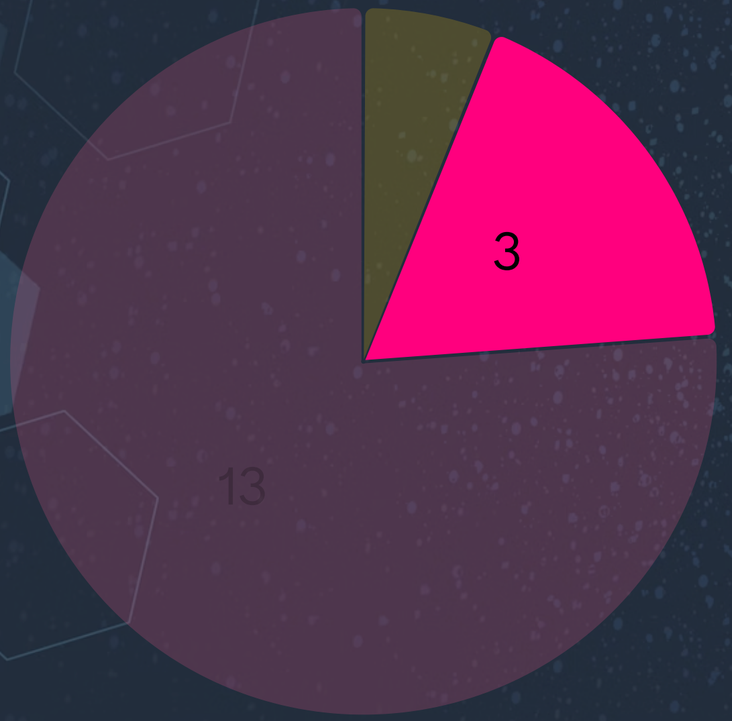
studying with adhd

IT PARK
UNIVERSITY

BREA

IT PARK UNIVERSITY

I am singing and doing squads - is it a concurrent activity or parallel?



1	concurrent	×
3	parallel	✓
13	concurrent-parallel	×

ITPU IT PARK UNIVERSITY

2. Thread Pools

A thread pool is a set of existing runnable (open) threads that are separated from the tasks of the Runnable/Callable type they perform and are used to perform several tasks at the same time.


Task Suppliers
Task Consumers
Task Queue

Thread pool workflow:

1. Task is specified in a Runnable
2. Runnable with a task is given to a pool of threads for processing
3. tasks are located in queue while threads are busy

Executors:

1. Executor is an interface that maintains the concurrent execution of tasks.
2. ExecutorService is an inheritor of the Executor interface; it adds methods that help control the lifecycles of the tasks and the executor itself.
3. ScheduledExecutorService is an inheritor of the ExecutorService interface; it manages delayed and/or periodic task execution.

Executor

This interface provides a single execute() method, which is a simple replacement for the low-level idiom for creation and execution of threads.

It is used to execute tasks of the Runnable type.

If threadRun is an object of the Runnable type and executorPool is an object of the Executor type, then:

- (new Thread(threadRun)).start(); is the creation and immediate execution of a thread (low level)
- executorPool.execute(threadRun); means either the same (a free existing runnable threadRun thread is used) or puts the task in a queue (which is more likely)

ExecutorService

This interface provides a universal method—submit()—to get and execute tasks of the Callable and Runnable types. The method returns an object of the Future type, which is used to get a returnable task value of the Callable type.

Also, it proposes a range of methods for managing Executor work:

- shutdown() waits for the execution of the tasks to be finished and terminates the executor.
- shutdownNow() terminates the executor immediately, interrupting the tasks being executed.

It allows task collections of the Callable type to be sent to execution:

- invokeAll() returns a list of results of all the tasks.
- invokeAny() returns the result of a task and cancels all tasks that have not been completed.

ScheduledExecutorService

This interface allows the start of task execution to be delayed for a specified period and task execution to be planned after a certain period. It contains the following methods:

- schedule() performs tasks of the Callable and Runnable types, which become available for execution after a delay.
- scheduleAtFixedRate() starts executing Runnable tasks after a specified delay and then repeats them after the given period. The process is terminated when the executor is stopped; the repeating interval does not include the time it takes to perform the task.
- scheduleWithFixedRate() starts executing Runnable tasks after a specified delay and then repeats them with the given delay between the termination of one execution and the commencement of the next.

ITPU IT PARK UNIVERSITY

ThreadpoolFactory

create thread pools

ThreadPoolDemo

tasks in pool

ThreadpoolFactory

Setting Up Pool Creation Manually

An Example of Setting Up a Thread Pool

Examine the step-by-step description of the code below:

Click the arrows to see more information.

Step 1

The class-task TaskAndThreadPool of the Runnable type is defined, which prints some information about itself into the console—the body of the run() method.

Step 2

A thread pool of the ExecutorService type is created in the main() method and configured as follows: The minimum number of working threads is 5; the maximum is 10; idle threads will exist for 30 seconds; tasks will be put in a queue of the LinkedBlockingQueue<Runnable> type.

Step 3

An array for storing tasks is created, and then the tasks themselves. Finally, they are sent into the thread pool by the execute() method to be executed.

Step 4

When the tasks are completed, the pool is terminated by calling the shutdown() method.

class TaskAndThreadPool

The Timer Class and the TimerTask Thread

Some kinds of tasks need to be executed **periodically**—for example, sending mail notifications, messages concerning password or license expiration, or reminders about regular payments like a subscription or maintenance fee. Threads are commonly used to execute tasks at a specific time or at fixed, regular intervals.

The following classes and methods are used to work with timer threads:

- Timer threads are described with the TimerTask abstract class in the util package implementing the Runnable interface.
- Thread execution is planned by the java.unit.Timer class, whose methods work with a TimerTask object.
- To execute threads, the schedule and scheduleAtFixedRate(params) methods of the Timer class are used.

TimerDemo

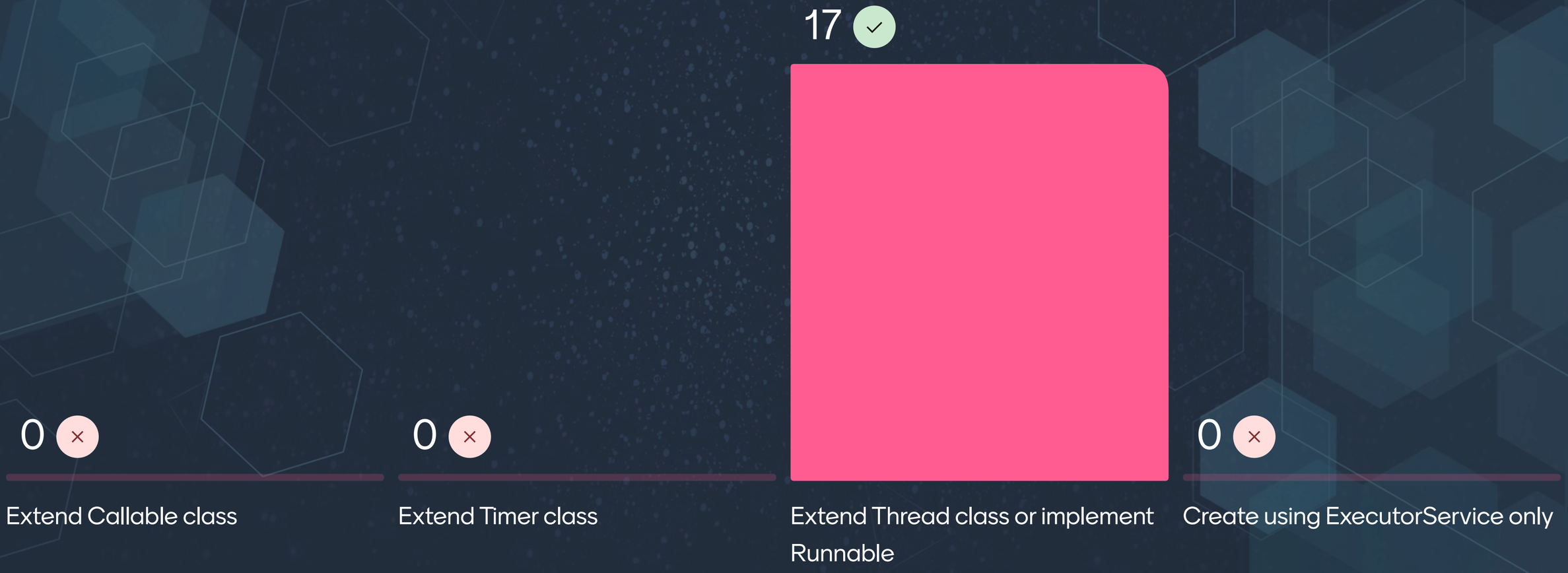
IT PARK
UNIVERSITY

QUIZ

Quiz time

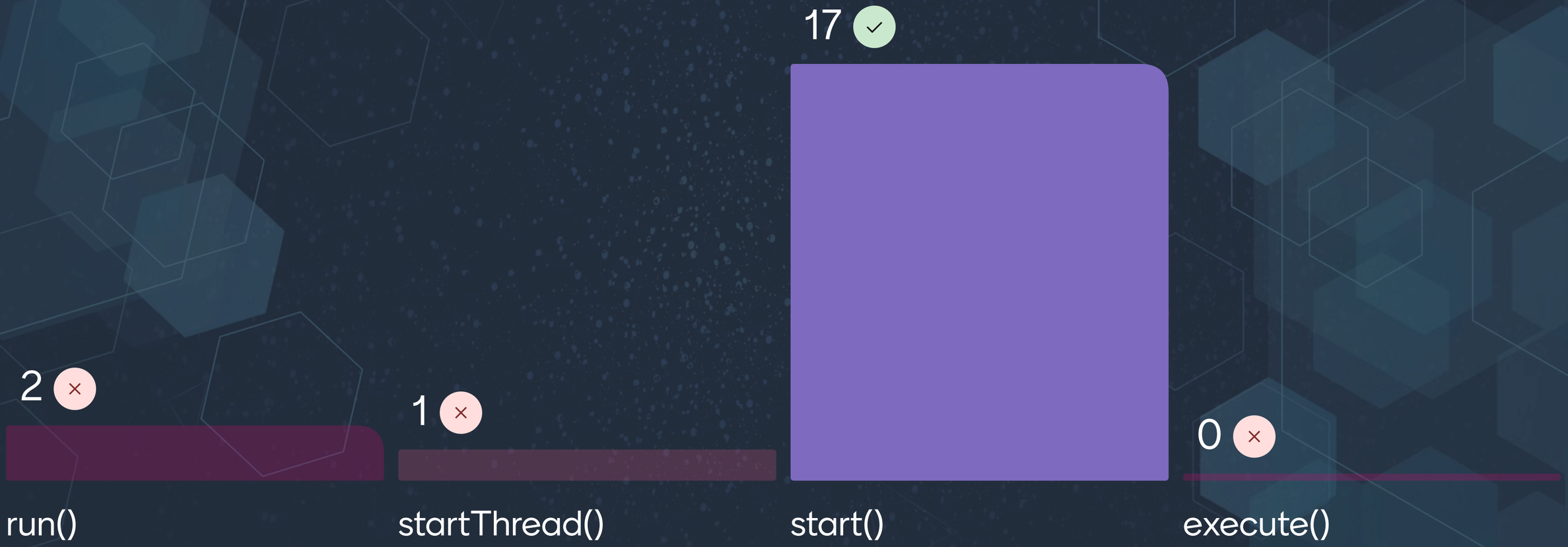
ITPU IT PARK UNIVERSITY

How do you create a thread in Java?



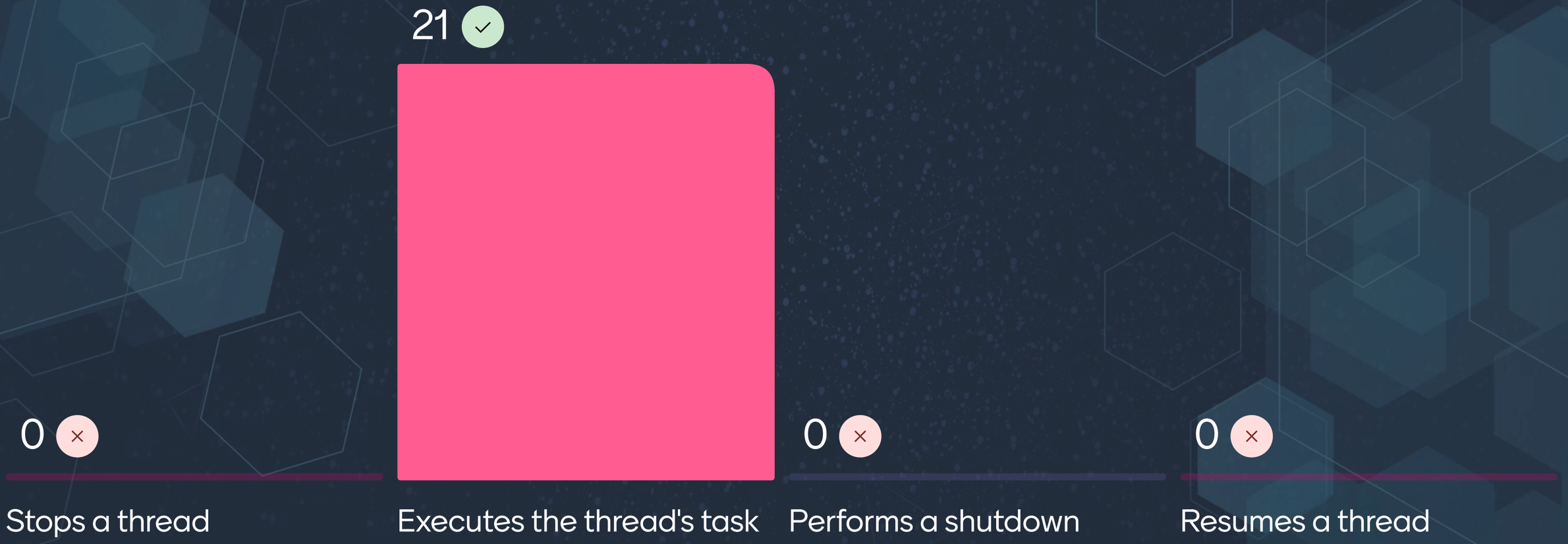
iTPU IT PARK UNIVERSITY

Which method is used to start a thread?



ITPU IT PARK UNIVERSITY

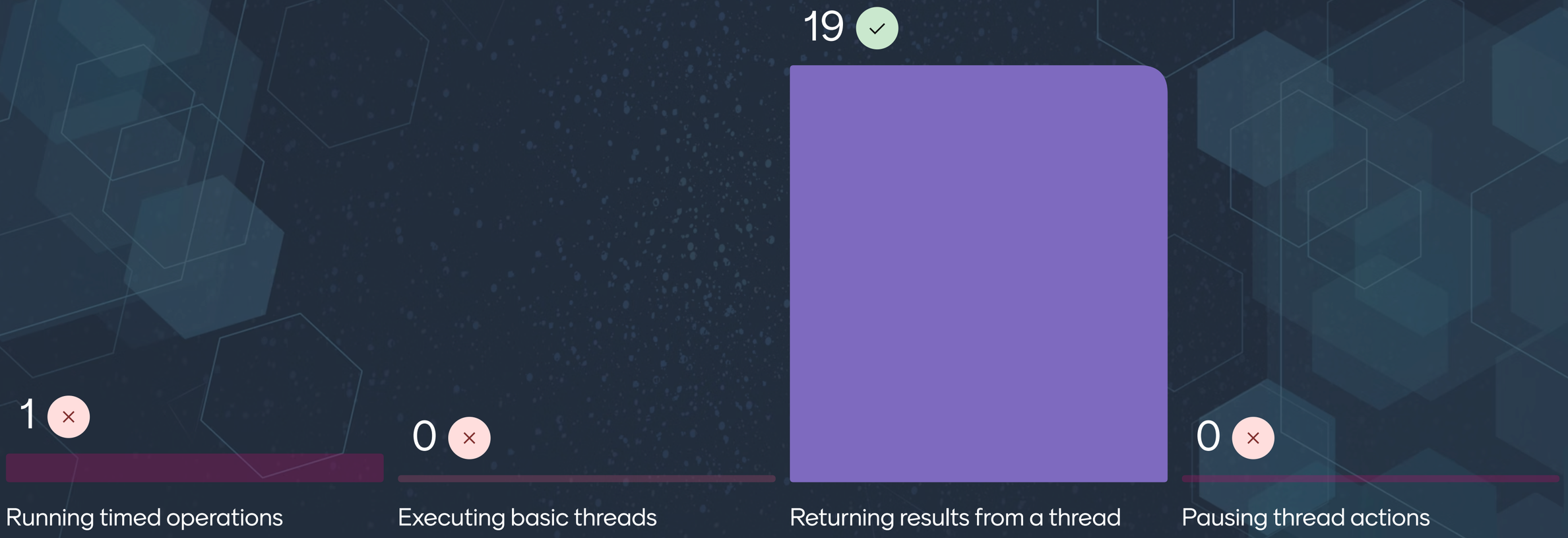
What does the run() method in a thread do?



1

iTPU IT PARK UNIVERSITY

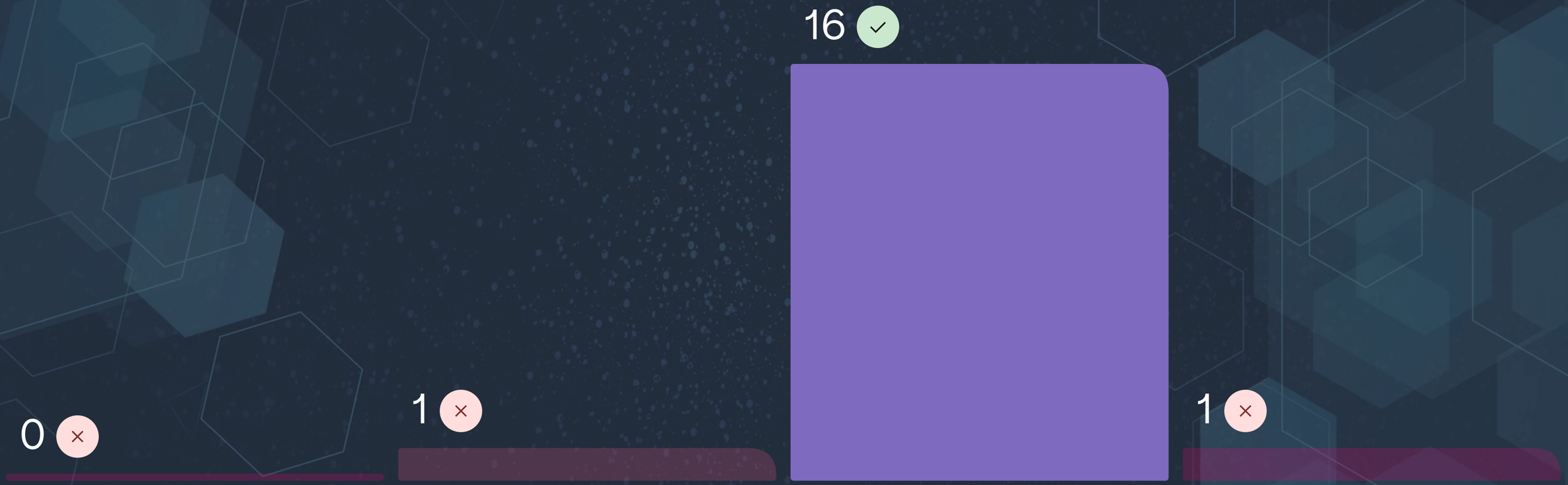
What is Callable used for in Java?



1

ITPU IT PARK UNIVERSITY

What is a Thread Pool?



A memory cache system

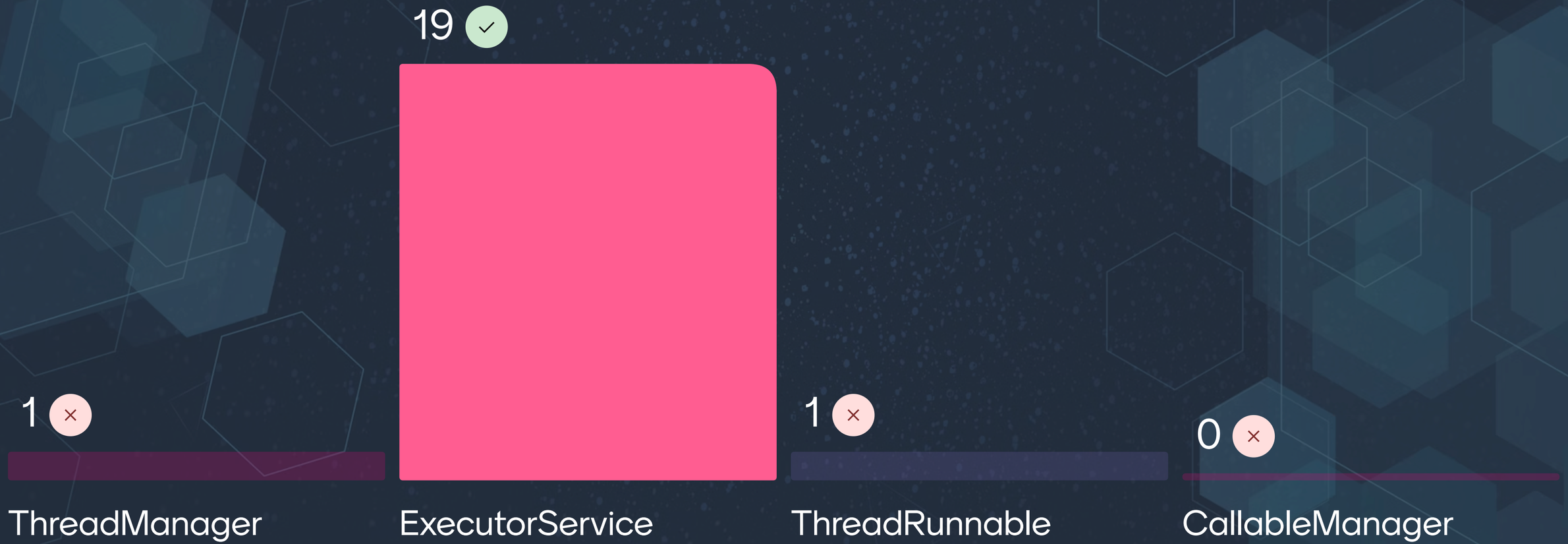
A collection of functions

A pool of reusable threads

Database connection manager

iTPU IT PARK UNIVERSITY

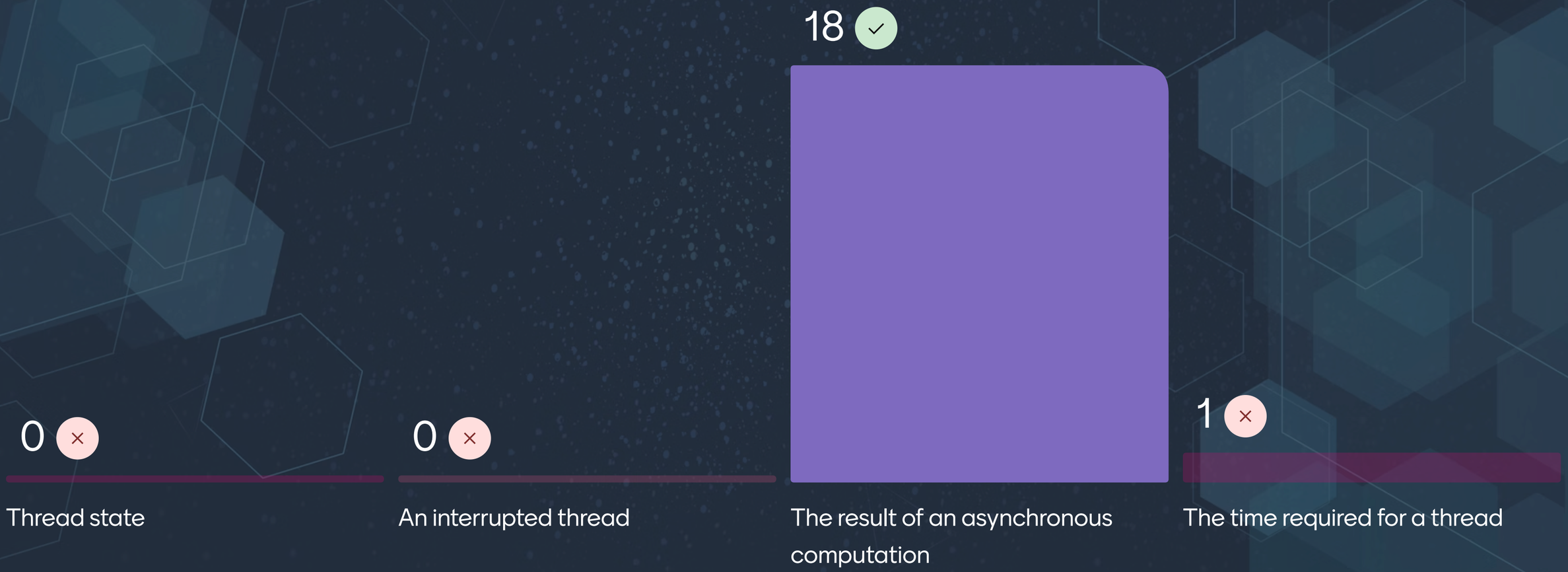
Which class provides thread pooling in Java?



1

ITPU IT PARK UNIVERSITY

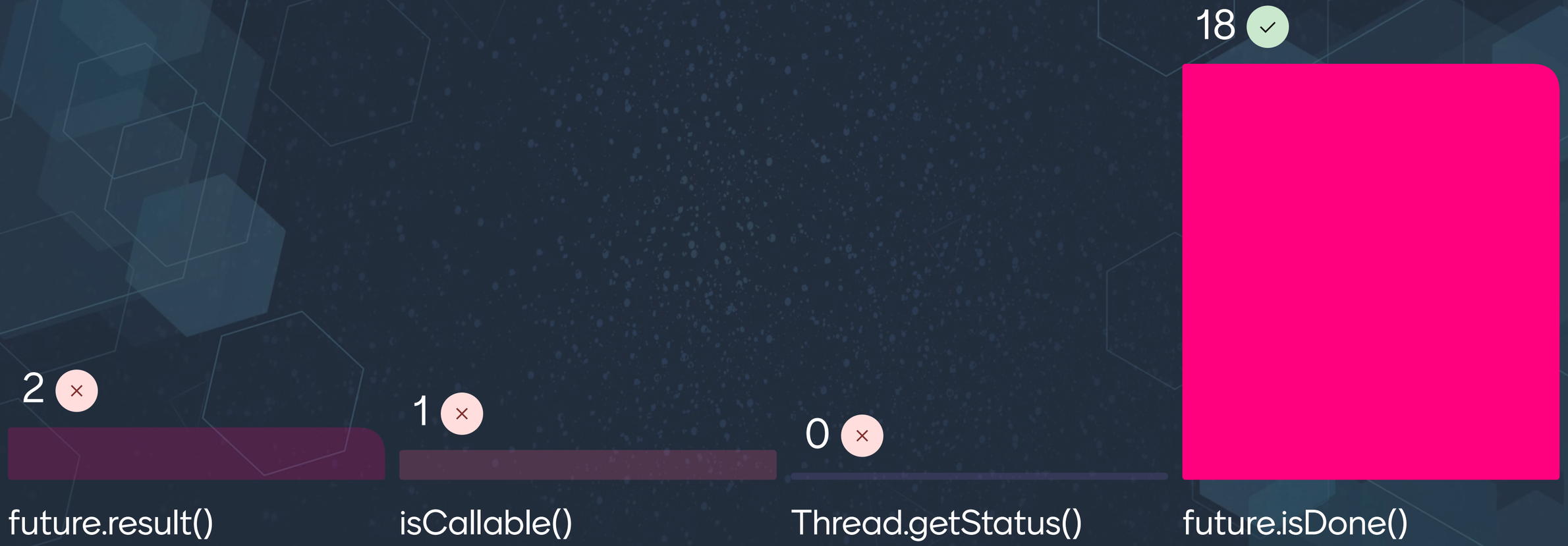
What does Future represent?



1

ITPU IT PARK UNIVERSITY

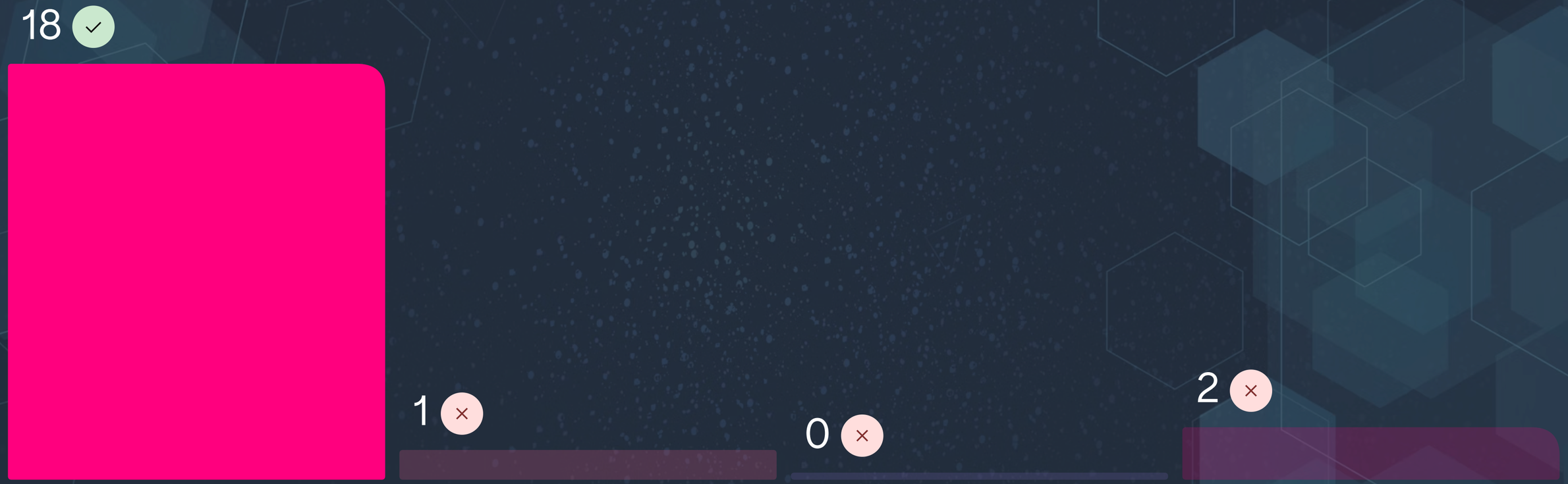
How do you check if a Future is done?



1

ITPU IT PARK UNIVERSITY

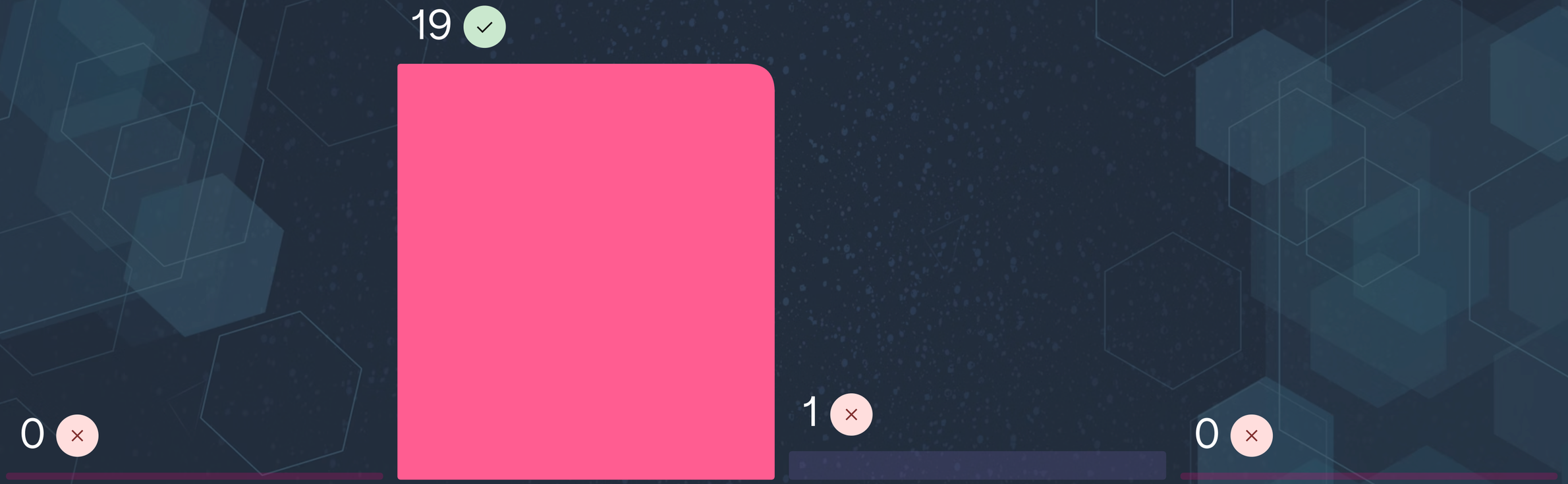
What does shutdown() in an ExecutorService do?



1

iTPU IT PARK UNIVERSITY

What is the purpose of join() in Java threads?



IT PARK UNIVERSITY

Quiz leaderboard

1

IT PARK UNIVERSITY

Q&A part

0 questions
0 upvotes